

Week 4- Sorting Algorithms I

Condensed Study Notes

Generated: 2026-05-25 10:44 UTC

Week 4- Sorting Algorithms I

Imagine you have a messy pile of books and you want to put them in alphabetical order on a shelf. Sorting algorithms are like different strategies or methods you could use to organize that pile of books.

This section is all about exploring some of these strategies for putting lists of things (like numbers or words) into a specific order, such as from smallest to largest or alphabetically. These ordered lists are much easier for computers to work with and understand.

We'll start by looking at one of the simplest ways to sort things, often called "Selection Sort".

Overview

We've already talked about sorting as a way to put things in order, like arranging books on a shelf. This section dives a bit deeper into what sorting actually means in the world of computers.

Imagine you have a list of numbers, say, [5, 2, 8, 1]. Sorting this list means rearranging these numbers so they follow a specific rule, like putting them from smallest to largest: [1, 2, 5, 8].

In computing, we call the things we're sorting "elements" or "items." They're usually kept together in a structure called an "array," which is like a numbered list or a row of boxes.

So, sorting is basically the process of "permuting" (which just means rearranging or reordering) these elements in an array according to a clear rule, like smallest to largest, or alphabetical order.

Why do we care about sorting so much? Because it's a really common and important step in many computer programs. Think of it as a crucial preparation step, like chopping vegetables before you cook them – it makes other tasks much easier to perform later on.

Categories of sorting

We can think about sorting in two main ways, depending on how much stuff we need to sort and where we're doing it. It's like organizing your bookshelf!

- **Internal sorting:** Imagine you have a small stack of books to sort on your desk. You can easily see and move all of them around right there on your desk. In computers, this is like when all the data you want to sort can fit into the computer's main workspace (called **memory**) all at once. It's quick because everything is readily available.
- **External sorting:** Now, picture having thousands of books to sort, way too many to fit on your desk. You'd probably sort them in batches, perhaps sorting a few shelves at a time and then combining the sorted sections. For computers, **external sorting** happens when there's too much data to fit into the main **memory** all at once. The computer has to use other storage places (like a hard drive, which is a

form of **external storage**) to hold the data and the sorted pieces. It sorts the data in chunks, saves those sorted chunks, and then combines them later. This is used when you have massive amounts of data.

Some sorting terminologies

We've talked about sorting as organizing data. Now, let's learn some of the specific words we use when talking about sorting algorithms.

In-place Sorting:

Imagine you have a messy bookshelf and you want to organize the books by author. An "in-place" way to do this would be to rearrange the books directly on the shelf itself, maybe by swapping books around. You wouldn't need to take all the books off the shelf and put them into a completely separate box to sort them first.

A sorting algorithm is called **in-place** if it sorts the data without needing a whole new, separate place to store the data while it's working. It might use a tiny bit of extra space for temporary notes (like a variable to hold a book you're about to swap), but it's mostly working directly on the original list. If an algorithm needs a lot of extra space to sort, it's called **not-in-place** or **out-of-place**.

Stable Sorting:

Let's say you have a list of students, and you want to sort them by their grade. If two students have the same grade, a **stable** sorting algorithm will keep them in the same order they were in originally. So, if Student A was listed before Student B in the original list, and they both get the same grade, Student A will still appear before Student B in the sorted list.

Essentially, a stable sort is careful not to mess with the order of items that are considered equal.

Sorting Order:

We sort things in a specific way. The most common ways are:

- **Numerical order:** Like sorting numbers from smallest to largest (e.g., 1, 2, 3) or largest to smallest (e.g., 3, 2, 1). This is also called **non-decreasing** (going up or staying the same) or **non-increasing** (going down or staying the same).
- **Lexicographical order:** This is like sorting words alphabetically. "Apple" comes before "Banana." It's the standard way we order text.

Sorting Problem & Algorithm:

A **sorting problem** is simply the task of putting a list of items into a specific order. A **sorting algorithm** is the set of step-by-step instructions that solves that problem – it's the method we use to do the sorting.

Common sorting algorithms

Now that we know what sorting is and some of its basic properties, we'll start looking at different ways (or 'algorithms') computers use to get lists organized.

Think of sorting algorithms like different methods for tidying up a messy pile of books. You could pick up one book at a time and put it exactly where it belongs, or you could have a more systematic approach.

We're going to explore six common sorting algorithms. We'll start with three that are considered 'elementary' because they are quite straightforward, though not always the most efficient for very large lists:

- **Selection Sort:** Imagine you have a pile of unsorted books, and you want to arrange them by size. With Selection Sort, you'd first find the smallest book and place it at the beginning. Then, from the remaining books, you'd find the next smallest and place it in the second spot, and so on. You 'select' the smallest remaining item and put it in its correct place.
- **Bubble Sort:** This is like repeatedly going through your list of books and comparing each book with the one next to it. If they are in the wrong order (e.g., a larger book is before a smaller one), you swap them. You keep doing this pass after pass, and with each pass, the 'heaviest' or 'largest' items 'bubble up' to their correct positions at the end of the list, much like bubbles rising in water.
- **Insertion Sort:** Think about sorting a hand of playing cards. You pick up one card at a time. As you pick up a new card, you insert it into the correct position within the cards you're already holding, which are already sorted. You build up your sorted list by taking items one by one and inserting them into their proper place in the already sorted portion.

Selection sort

Imagine you have a messy pile of books and you want to arrange them by size, from smallest to largest. Selection Sort is like picking up the smallest book from the entire pile, putting it in its correct spot, then looking at the remaining books and picking the next smallest, and so on.

Selection Sort is a way to organize a list (like our books) by repeatedly finding the smallest item that hasn't been sorted yet and putting it in the right place.

Here's how it works, step-by-step:

1. **Find the smallest:** Look through all the items in the list that haven't been sorted yet and find the very smallest one.
2. **Put it in place:** Take that smallest item and move it to the very beginning of the unsorted part of the list. This part is now considered sorted.
3. **Repeat:** Do this all over again for the remaining unsorted items until everything is in order.

It's called a 'brute-force' approach because it's straightforward and doesn't try any clever shortcuts; it just systematically finds the smallest item each time.

Selection sort algorithm

Imagine you have a messy pile of papers, and you want to arrange them alphabetically. Selection Sort is like going through that pile, finding the very first paper alphabetically, and setting it aside as the start of your sorted stack. Then, you look through the *remaining* messy papers to find the *next* one alphabetically, and add it to your sorted stack. You keep doing this – finding the smallest item from what's left and moving

it to the end of your sorted section – until all the papers are in order.

Here's how this works in a computer:

- **Finding the smallest:** The algorithm looks at all the items in the list that haven't been sorted yet.
- **Swapping:** It finds the smallest item among those unsorted items and swaps it with the item that's currently at the beginning of the unsorted section. This effectively moves the smallest item to its correct, sorted position.
- **Shrinking the unsorted part:** Once an item is placed in its sorted spot, the 'unsorted' part of the list gets smaller by one item, and the algorithm repeats the process for the remaining unsorted items.
- **Completion:** This continues until the entire list is sorted.

The selection sort is used when

Selection Sort is a good choice in a few specific situations.

Imagine you have a small stack of papers to sort, and you don't mind moving them around a lot. Selection Sort is like that.

Here's when it's a good fit:

- **When you need to look at every single item:** Selection Sort always checks all the items in the list to find the smallest one. So, if your task absolutely requires a thorough check of everything, it's suitable.
- **For small lists:** If you only have a few items to sort, the extra effort of checking everything isn't a big deal, and Selection Sort works efficiently.
- **When moving items around is cheap:** Sometimes, the cost or effort of 'swapping' (moving an item from one place to another) doesn't matter much. Selection Sort does a number of swaps, but if those swaps are easy, it's fine.
- **When writing to memory is expensive:** Think of certain types of memory, like flash memory (used in USB drives or SSDs), where writing data is more costly or wears out the memory faster than reading it. Selection Sort performs a number of 'writes' (which are like swaps) that is related to the size of the list (we say this cost is $O(n)$). If writing is a bottleneck, and you're sorting a small list, Selection Sort might be better because it minimizes writes compared to some other sorting methods.

Bubble sort

Imagine you have a line of people who are all trying to arrange themselves by height, from shortest to tallest. Bubble sort is like going down the line and asking each pair of people next to each other, 'Are you in the right order?' If the person on the left is taller than the person on the right, you ask them to swap places. You keep doing this all the way down the line, and then you start over from the beginning again. With each pass, the tallest person 'bubbles up' to their correct spot at the end of the line.

Here's how it works in terms of lists:

- **Comparing Neighbors:** Bubble sort looks at two items in a list that are right next to each other. These are called **adjacent elements** (meaning they are side-by-side).

- **Swapping if Needed:** If these two adjacent elements are in the wrong order (for example, if you're sorting numbers from smallest to largest and the first number is bigger than the second), the algorithm swaps their positions.
- **The 'Bubbling Up' Effect:** By repeatedly comparing and swapping adjacent elements, the largest (or smallest, depending on the sorting order) element in the unsorted part of the list gradually moves towards its correct final position, much like a bubble rising to the surface of water.
- **Repeating the Process:** This whole process of comparing and swapping adjacent elements is done over and over again until the entire list is sorted.

Bubble sort algorithm

Imagine you have a line of people, and you want them to stand in order of height, shortest to tallest. Bubble Sort is like going through the line, comparing each person to the one next to them, and if the person on the left is taller than the person on the right, you swap their places. You keep doing this over and over, and with each pass, the tallest people gradually move to the end of the line, like bubbles rising to the surface.

Here's how it works:

- **Repeated Comparisons and Swaps:** Bubble Sort repeatedly goes through the list, comparing adjacent items (items right next to each other).
- **'Bubbling Up' Larger Items:** If two adjacent items are in the wrong order (e.g., the first is larger than the second when you want smallest to largest), they are swapped. This action causes larger items to 'bubble up' towards their correct position at the end of the list.
- **Completion:** This process continues until a full pass through the list is made with no swaps, meaning all items are now in their correct order.

The bubble sort is used when

Bubble sort is a straightforward sorting method. It's a good choice when you need code that is easy to write and understand, and when the efficiency of the sorting process itself isn't a major concern.

Think about organizing a small pile of papers for a simple task. You might just go through them, making sure they're in order by quickly glancing and swapping any that are out of place. You're not worried about doing it in the absolute fastest way possible, just that it gets done correctly and easily.

So, Bubble Sort is useful in these situations:

- When you want to write code that is short and simple.
- When the complexity of the sorting process (how fast or slow it is) doesn't really matter for your specific task.

Insertion sort

Imagine you're sorting a hand of playing cards. You pick up one card at a time from a messy pile and insert it into the correct spot in the cards you're already holding, which are already sorted.

Insertion sort works similarly. It takes elements from an unsorted part of a list one by one and finds the perfect place for them within the part of the list that's already sorted. It keeps doing this until the entire list is organized.

This method uses a strategy called 'decrease and conquer'. Think of it like solving a big puzzle by first solving smaller, related puzzles. With insertion sort, we assume that a smaller version of the list is already sorted. Then, we take the next unsorted item and carefully slot it into its right place among those already sorted items.

Insertion sort algorithm

Imagine you're sorting a hand of playing cards. You pick up one card at a time from a messy pile and then find the right spot to slide it into your already sorted hand. That's basically how Insertion Sort works!

Insertion Sort builds a sorted list one item at a time. It takes an item from the unsorted part of the list and inserts it into its correct place within the part of the list that's already sorted.

Here's how it goes step-by-step:

- You start with an empty sorted list (or the first item is considered sorted by itself).
- You pick the next item from the unsorted part.
- You compare this item with the items in the sorted part, moving backward, until you find the spot where it belongs.
- You slide the item into that spot, shifting other items over if needed to make space.
- You repeat this until all items have been moved from the unsorted part to the sorted part.

Think of it like organizing books on a shelf. You pick up a book from a stack and then find where it should go on the shelf among the books already arranged. You might have to move some books over to make room for the new one.

Insertion Sort is used when

Insertion sort is a handy sorting method that shines in a couple of specific situations.

Think of it like tidying up a small stack of papers on your desk versus organizing a massive library. Insertion sort is much better suited for the smaller task.

Key situations where Insertion Sort is a good choice:

- **When you only have a tiny number of items left to sort:** Imagine you've already sorted most of your books, and only a few are left out. Insertion sort is excellent at placing those last few items into their correct spots.
- **When the entire list you're working with is already small:** If you have a very short list of things to organize from the start, insertion sort is an efficient way to get it done.

Summary of complexities of the elementary sorting algorithms

Now that we've explored Bubble Sort, Selection Sort, and Insertion Sort, let's look at how we measure how 'good' they are. Think of it like comparing different ways to clean your house: some methods might be faster overall, but take more effort in certain situations, or use more 'space' (like needing a big bucket).

We'll look at:

- **Time Complexity:** How much time the algorithm takes to run, usually measured by how many steps it performs as the list size grows.
- **Overall:** This is a general idea of how fast the algorithm is for most cases. For Bubble Sort, Selection Sort, and Insertion Sort, the overall time complexity is often described as $O(n^2)$ (read as 'Oh of n squared'). This means if you double the number of items in your list, the time it takes might go up by about four times. It's like cleaning a room that's twice as big – it takes much more than twice the time.
- **Best Case:** This is the fastest scenario for an algorithm. For Bubble Sort and Insertion Sort, the best case can be as good as $O(n)$ (read as 'Oh of n'), meaning the time grows directly with the list size. Imagine a list that's already sorted – Insertion Sort can finish very quickly!
- **Average Case:** This is what you'd expect most of the time. For all three, the average case is generally $O(n^2)$.
- **Worst Case:** This is the slowest scenario. For all three algorithms, the worst case is also $O(n^2)$.
- **Swaps or Movements:** This measures how many times items in the list are moved around.
- **Best Case:** For all three, the best case for swaps is very low, often just 1 ($O(1)$).
- **Worst Case:** For Bubble Sort and Insertion Sort, the worst case can be $O(n^2)$, meaning many items might need to be moved. For Selection Sort, the worst case is better at $O(n)$, as it only moves items once they are found to be the smallest.
- **Auxiliary Space:** This is the extra memory an algorithm needs to do its work. All three elementary sorts are great here because they use very little extra memory, typically $O(1)$ (read as 'Oh of 1'), meaning they don't need much more space regardless of how big the list is. They are 'in-place' sorts.
- **Algorithmic Paradigm:** This describes the general strategy the algorithm uses.
 - Bubble Sort and Selection Sort are considered 'Brute Force' methods, meaning they systematically check possibilities.
 - Insertion Sort uses a 'Decrease and Conquer' approach, solving smaller parts of the problem first.
- **Sorting Method:** How they actually rearrange the data.
 - Bubble Sort and Selection Sort primarily use 'Swapping' to move items.
 - Insertion Sort uses 'Shifting' to make space for an item.
- **Sorting in Place?:** Yes, all three of these are 'in-place' sorts, meaning they sort the list directly without needing a lot of extra memory to hold a second copy.
- **Stability:** This tells us if the algorithm keeps items with the same value in their original relative order.
 - Bubble Sort and Insertion Sort are 'Stable'. If you have two '5's and the first one appeared earlier in the original list, it will still appear before the second '5' after sorting.

- Selection Sort is 'Not Stable'. The original order of equal items might change.